

Announcements

If you joined the class late, please see:

- <https://sp26.datastructur.es/policies/#late-adding-cs61b> (will be up by ~3 PM today)

For E7 students: Turns out that recent editions don't cover classes or maps.

- Sorry, that might have made worksheet 1 very scary!
- Today and next Monday will go through these topics in some detail.

HW1 and HW2 are due today.

- But if you turn them in late, you only lose 10% points per day and can make up for them later.
- If you joined the class late, see the link above for how to avoid the late penalty.

Why bother going to lecture?

- It's fun to show up. Didn't show up I'd just be chilling.
- Keeps me organized, stay on track.
- Random objects! I need these things.
- Get to see Josh Hug AND Kay.
- We pay to be here, so I want the goodies for which I pay.
- I want to feel like I'm in college.
- Yell things and everyone has to listen.
- Make sure I get out of my dorm.
- Watching lecture videos is less compelling.

There will be exam recovery points given for attending lecture.

- Value of an exam recovery point decreases with your paper exam score, for example:
 - If you earn 0% on paper exams, each recovery point will count for one point towards the course.
 - If you have 35% on paper exams, each recovery point counts for 1/2 a point.
 - If you have 70% or more on paper exams, recovery points don't count.
- Target mean for paper exams: 65%.

Also note: Policies page coming soon.

- Sorry, still ironing out details around our many new logistical changes, especially the CBTF.

Further Notes for Webcast Viewers

- Any time I'm live coding, I advise you to pause frequently and try to anticipate my next move. You'll probably learn more by trying to guess what I'm going to do rather than just watching me do it.





Lecture 2

Defining Classes. Lists and Arrays.

CS61B, Spring 2026 @ UC Berkeley

Josh Hug and Kay Ousterhout

Defining and Instantiating Classes

Lecture 2, CS61B, Spring 2026

Classes in Java

- **Defining and Instantiating Classes**
- Class Terminology
- Static vs. Instance Members
- `public`
- `public static void main(String[] args)`

Lists and Arrays

- Lists in Java 4.0
- Abstract Data Types vs. Concrete Implementations

Classes in Python

Just like Python, Java supports the ability for you to define your own types (classes).

```
class Dog():  
    def make_noise(self):  
        print("bark")  
  
d = Dog()  
d.make_noise()
```

```
class Dog {  
    void makeNoise() {  
        System.out.println("bark");  
    }  
  
    void main() {  
        Dog d = new Dog();  
        d.makeNoise();  
    }  
}
```

Note: Can technically put `void main()` outside of the class, but I'll keep it inside. More later.

Classes in Python

Just like Python, Java supports the ability for you to define your own types (classes).

```
class Dog():  
    def make_noise(self):  
        print("bark")  
  
d = Dog()  
d.make_noise()
```

```
class Dog {  
    void makeNoise() {  
        System.out.println("bark");  
    }  
  
    void main() {  
        Dog d = new Dog();  
        d.makeNoise();  
    }  
}
```

Note: Can technically put `void main()` outside of the class, but I'll keep it inside. More later.

Not all dogs are equal!



Classes, Instance Variables, and Constructors (in Python)

As you've hopefully seen in a previous class, instances of objects can contain data. Example:

```
class Dog():
    def __init__(self, size):
        self.size = size

    def make_noise(self):
        if self.size < 10:
            print("yip!")
        elif self.size < 30:
            print("bark.")
        else:
            print("wooooooof!")

maya = Dog(100)
maya.make_noise()
```

[Demo](#)

Classes, Instance Variables, and Constructors (in Java)

Java classes are similar. Let's create an equivalent class in Java.

```
class Dog():
    def __init__(self, size):
        self.size = size

    def make_noise(self):
        if self.size < 10:
            print("yip!")
        elif self.size < 30:
            print("bark.")
        else:
            print("wooooooof!")

maya = Dog(100)
maya.make_noise()
```

Classes, Instance Variables, and Constructors (in Java)

What differences do you notice?

```
class Dog():
    def __init__(self, size):
        self.size = size

    def make_noise(self):
        if self.size < 10:
            print("yip!")
        elif self.size < 30:
            print("bark.")
        else:
            print("wooooooof!")

maya = Dog(100)
maya.make_noise()
```

```
class Dog {
    int size;

    Dog(int s) {
        size = s;
    }

    void makeNoise() {
        if (size < 10) {
            IO.println("yipyipyip!");
        } else ...
    }

    void main() {
        Dog maya = new Dog(100);
        maya.makeNoise();
    }
}
```

Class Terminology

Lecture 2, CS61B, Spring 2026

Classes in Java

- Defining and Instantiating Classes
- **Class Terminology**
- Static vs. Instance Members
- public
- public static void main(String[] args)

Lists and Arrays

- Lists in Java 4.0
- Abstract Data Types vs. Concrete Implementations

Defining a Typical Class (Terminology)

```
class Dog {  
    int size;  
  
    Dog(int s) {  
        size = s;  
    }  
  
    void makeNoise() {  
        if (size < 10) {  
            System.out.println("yipyipyip!");  
        } else if (size < 30) {  
            System.out.println("bark. bark.");  
        } else {  
            System.out.println("woof!");  
        }  
    }  
}
```

Instance variable. Can have as many of these as you want.

Constructor (similar to a method, but not a method). Determines how to instantiate the class.

Instance Method.

- A function that is part of a class is often called a “method.”
- We will see that in Java, there is another kind of method called a “static method”.

Instance Variables in Java

Classes provide a blueprint for instances of that class.

These instances are
also called 'objects'

Dog.java provides a blueprint that all Dog objects.

- For the example above, all Dog objects will have exactly one variable, of type int, called size.
- **Cannot add new instance variables to a Dog. They must ALL obey the blueprint exactly.**

```
class Dog {  
    int size;
```

```
Dog hugeDog = new Dog(150);  
hugeDog.size = 5;           // size guaranteed to exist  
hugeDog.name = "frank";    // will crash, name doesn't exist
```

Instantiating a Class and Terminology

```
void main(String[] args) {  
    Dog smallDog;  
    new Dog(20);  
    smallDog = new Dog(5);  
    Dog hugeDog = new Dog(150);  
    smallDog.makeNoise();  
    hugeDog.makeNoise();  
}
```

Declaration of a Dog variable.

Instantiation of the Dog class as a Dog Object.

Instantiation and Assignment.

Declaration, Instantiation and Assignment.

Invocation of the 150 lb Dog's makeNoise method.

The dot notation means that we want to use a method or variable belonging to hugeDog, or more succinctly, a **member** of hugeDog.

Static vs. Instance Members

Lecture 2, CS61B, Spring 2026

Classes in Java

- Defining and Instantiating Classes
- Class Terminology
- **Static vs. Instance Members**
- `public`
- `public static void main(String[] args)`

Lists and Arrays

- Lists in Java 4.0
- Abstract Data Types vs. Concrete Implementations

Accessing Classes from Other .java Files

Earlier, we had a main method in the Dog class itself.

- But other code could use the Dog class as well. Example below.

Dog.java

```
class Dog {
    int size;

    Dog(int s) {
        size = s;
    }

    void makeNoise() {
        if (size < 10) {
            System.out.println("yipypypip!");
        } else if (size < 30) {
            System.out.println("bark. bark.");
        } else {
            System.out.println("woof!");
        }
    }
}
```

DogInvestigator.java

```
void main() {
    Dog lilDog = new Dog(3);
    Dog clifford = new Dog(1000);

    clifford.makeNoise();
}
```

Note: Unlike Python, you don't need to import code from other .java files!

Java automatically scans all folders from a list to see if the desired class exists.

Adding a New Method

Let's expand the capability of our Dog class so that it can compare dogs.

- Goal: The code on the right should work.
- maxDog should be a function that returns the larger of current dog and other dog.

DogInvestigator.java

```
void main() {  
    Dog lilDog = new Dog(3);  
    Dog clifford = new Dog(1000);  
  
    Dog bigger = lilDog.maxDog(clifford);  
    bigger.makeNoise();  
}
```

Adding a New Method

Let's expand the capability of our Dog class so that it can compare dogs.

- Goal: The code on the right should work.
- maxDog should be a function that returns the larger of current dog and other dog.

Dog.java

```
class Dog {  
    int size;  
  
    Dog(int s) {  
        size = s;  
    }  
  
    ...  
  
    Dog maxDog(Dog otherDog) {  
        if (otherDog.size > this.size) {  
            return otherDog;  
        }  
        return this;  
    }  
}
```

DogInvestigator.java

```
void main() {  
    Dog lilDog = new Dog(3);  
    Dog clifford = new Dog(1000);  
  
    Dog bigger = lilDog.maxDog(clifford);  
    bigger.makeNoise();  
}
```

This **this** is optional.

This **this** is required.

Alternate Approach

It is also possible to define the method such that the class itself does the comparison.

- That is: It's not lilDog or Clifford doing the comparison, it's Dog.

Let's see how we'd do this.

DogInvestigator.java

```
void main() {  
    Dog lilDog = new Dog(3);  
    Dog clifford = new Dog(1000);  
  
    Dog bigger = Dog.maxDog(lilDog, clifford);  
    bigger.makeNoise();  
}
```

Alternate Approach

It is also possible to define the method such that the class itself does the comparison.

- That is: It's not lilDog or Clifford doing the comparison, it's Dog.

Dog.java

```
class Dog {
    int size;

    Dog(int s) {
        size = s;
    }

    ...

    static Dog maxDog(Dog d1, Dog d2) {
        if (d1.size > d2.size) {
            return d1;
        }
        return d2;
    }
}
```

DogInvestigator.java

```
void main() {
    Dog lilDog = new Dog(3);
    Dog clifford = new Dog(1000);

    Dog bigger = Dog.maxDog(lilDog, clifford);
    bigger.makeNoise();
}
```

Alternate Approach

It is also possible to define the method such that the class itself does the comparison.

- That is: It's not lilDog or Clifford doing the comparison, it's Dog.

Dog.java

```
class Dog {
    int size;

    Dog(int s) {
        size = s;
    }

    ...

    static Dog maxDog(Dog d1, Dog d2) {
        if (d1.size > d2.size) {
            return d1;
        }
        return d2;
    }
}
```

DogInvestigator.java

```
void main() {
    Dog lilDog = new Dog(3);
    Dog clifford = new Dog(1000);

    Dog bigger = Dog.maxDog(lilDog, clifford);
    bigger.makeNoise();
}
```

By declaring the method “static”, we are saying:

- This method is invoked using the class name.
- There is no **this** in the context of this method.

Suppose we have the class shown:



```
class Human {  
    int consider(int x) {  
        ...  
    }  
  
    static int ponder(int y) {  
        ...  
    }  
}
```

Which of the following usages is appropriate? www.yellkey.com/book

```
Human h = new Human();  
h.consider(5);  
Human.consider(10);  
h.ponder(10);  
Human.ponder(10);
```


Check Your Understanding #1

Suppose we have the class shown:

```
class Human {  
    int consider(int x) {  
        ...  
    }  
  
    static int ponder(int y) {  
        ...  
    }  
}
```

Which of the following usages is appropriate?

```
Human h = new Human();
```

```
h.consider(5);
```

```
Human.consider(10);
```

← Compile error

```
h.ponder(10);
```

← Arguably should be
a compile error

```
Human.ponder(10);
```

Check Your Understanding #1

Suppose we have the class shown:

```
class Human {  
    int consider(int x) {  
        ...  
    }  
  
    static int ponder(int y) {  
        ...  
    }  
}
```

Suppose we have the following. What do the two bottom calls return?

```
Human h = new Human();  
h.consider(5);    // returns 5000  
Human.ponder(10); // returns 1000  
h.ponder(10);     // returns ?  
Human.ponder(10); // returns ?
```

Static vs. Non-Static Methods Summary

Key differences between static and non-static (a.k.a. instance) methods:

- Static methods are invoked using the class name, e.g. `Dog.makeNoise()`;
- Instance methods are invoked using an instance name, e.g. `maya.makeNoise()`;
- Static methods can't access "my" instance variables, because there is no "me".

Static

```
static void makeNoise() {  
    System.out.println("Bark!");  
}
```

This method cannot access size! Doesn't exist.

Invocation:

```
Dog.makeNoise();
```

Non-static

```
void makeNoise() {  
    if (size < 10) {  
        System.out.println("yipyipyip!");  
    } else if (size < 30) {  
        System.out.println("bark. bark.");  
    } else { System.out.println("woof!"); }  
}
```

Invocation:

```
Dog maya = new Dog(100);  
maya.makeNoise();
```

Why Static Methods?

Some classes are never instantiated. For example, Math.

- `x = Math.round(5.6);`

← Much nicer than:

```
Math m = new Math();  
x = m.round(x);
```

Or in the case of our Dog class, maybe we just liked it aesthetically if we had the class itself do the comparing.

Static Variables (are Dangerous)

Classes can also have static variables.

- You should always access class variables using the class name, not an instance name.
 - Bad coding style to do something like `maya.binomen`.
 - Even worse to do something like `maya.binomen = "Vulpes vulpes"`
- **Warning: Strongly recommended to avoid static variables whose values change.**
 - Leads to complicated code: Becomes hard to mentally keep track of which parts of your program read and write from/to the static variable. For more [read this](#).

```
class Dog {  
    int size;  
    static String binomen = "Canis familiaris";  
  
    Dog(int s) {  
        size = s;  
    }  
    ...  
}
```

Static vs. Non-Static

A class may have a mix of static and non-static members.

- A variable or method defined in a class is also called a member of that class.
- Static members are accessed using class name, e.g. Dog.binomen.
- Non-static members cannot be invoked using class name: ~~Dog.makeNoise()~~
- Static methods must access instance variables via a specific instance, e.g. d1.

```
class Dog {
    int size;
    static String binomen = "Canis familiaris";

    Dog(int s) {
        size = s;
    }

    static Dog maxDog(Dog d1, Dog d2) {
        if (d1.size > d2.size)
            { return d1; }
        return d2;
    }
    ...
}
```

```
...
void makeNoise() {
    if (size < 10) {
        System.out.println("yipyipyip!");
    } else if (size < 30) {
        System.out.println("bark. bark.");
    } else {
        System.out.println("woof!");
    }
}
}
```

public

Lecture 2, CS61B, Spring 2026

Classes in Java

- Defining and Instantiating Classes
- Class Terminology
- Static vs. Instance Members
- **public**
- `public static void main(String[] args)`

Lists and Arrays

- Lists in Java 4.0
- Abstract Data Types vs. Concrete Implementations

The public Keyword

Often, you'll see that in real world code, classes, constructors, variables, and methods are preceded by the keyword **public**, e.g.

- We'll talk a bit more about this in lecture 3.
- For now, it makes no difference if you include this keyword or not.

```
public class Dog {  
    public int size;  
    public static String binomen = "Canis familiaris";  
  
    public Dog(int s) {  
        size = size;  
    }  
  
    public static Dog maxDog(Dog d1, Dog d2) {  
        if (d1.size > d2.size)  
            { return d1; }  
        return d2;  
    }  
    ...  
}
```


public static void main(String[] args)

Lecture 2, CS61B, Spring 2026

Classes in Java

- Defining and Instantiating Classes
- Class Terminology
- Static vs. Instance Members
- public
- **public static void main(String[] args)**

Lists and Arrays

- Lists in Java 4.0
- Abstract Data Types vs. Concrete Implementations

Before September 2025:

- All Java code had to be part of a class.
- Printing was done by writing `System.out.println` instead of `IO.println`.
- The main method had to be declared as `public static void main(String[] args)` instead of just `void main()`.

Compare our Spring 2026 HelloWorld with the Fall 2025 semester's HelloWorld.

HelloWorld.java (sp26)

```
void main() {  
    IO.println("hello world");  
}
```

HelloWorld.java (fa25)

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("hello world");  
    }  
}
```

Before September 2025:

- All Java code had to be part of a class.
- Printing was done by writing `System.out.println` instead of `IO.println`.
- The main method had to be declared as `public static void main(String[] args)` instead of just `void main()`.

When you look at resources for this class that are from before this semester, you may see this! Don't be surprised.

- Also oops: I forgot to change HW1 and HW2 to use the simple Lecture 1 syntax. Sorry.

HelloWorld.java (sp26)

```
void main() {  
    IO.println("hello world");  
}
```

HelloWorld.java (fa25)

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("hello world");  
    }  
}
```

Why `public static void main(String[] args)`? Well:

- This is a public method (more in lecture 3).
- It is static (doesn't require instantiating the class).
- It returns nothing (so we say void).
- Its name is main.
- **Beyond scope of class:** It takes an array of Strings as an argument.
 - Passed by your operating system if you run from the command line.
 - Example, imagine you have a program that morphs two images called `Morph.java`. You'd run it as e.g. `java Morph joshhug.jpg kayousterhout.jpg`, and those two strings would be given as arguments to your program.

HelloWorld.java (sp26)

```
void main() {  
    IO.println("hello world");  
}
```

HelloWorld.java (fa25)

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("hello world");  
    }  
}
```

Lists in Java 4.0

Lecture 2, CS61B, Spring 2026

Classes in Java

- Defining and Instantiating Classes
- Class Terminology
- Static vs. Instance Members
- `public`
- `public static void main(String[] args)`

Lists and Arrays

- **Lists in Java 4.0**
- Abstract Data Types vs. Concrete Implementations

Lists

In programming languages, a list is an ordered sequence of objects, often represented by comma-separated values in-between brackets.

Example: [3, 6, 9, 12, 15]

Lists support a variety of operations which vary according to the whims of the authors who wrote the code for the list. Some examples:

- Append an item, e.g. we could append 18 to the list above.
- Retrieve an item by index, e.g. we could ask for the 0th item and get 3.
- Removing an item by index or value, e.g. we could remove the 9 from the list.

For more, see wikipedia: [https://en.wikipedia.org/wiki/List_\(abstract_data_type\)](https://en.wikipedia.org/wiki/List_(abstract_data_type))

Lists in Python

Python lists have very simple syntax, given below.

```
L = []  
L.append("a")  
L.append("b")  
L.append("c")  
print(L)
```

```
['a', 'b', 'c']
```

Let's try to make a copy of the code below using IntelliJ.

- See the recording for more discussion. These slides will be minimalist.

```
L = []  
L.append("a")  
L.append("b")  
L.append("c")  
print(L)
```

```
['a', 'b', 'c']
```


Lists in Java Attempt #1

Let's try to make a copy of the code below using IntelliJ.

- The Java code below won't compile.
- IntelliJ gives the error "can't resolve symbol List".
- The fix is to import List.

```
L = []  
L.append("a")  
L.append("b")  
L.append("c")  
print(L)
```

```
['a', 'b', 'c']
```

```
void main() {  
    List<String> L = new List();  
}
```

Can either add import statement to code, or use the IntelliJ option-enter or alt-enter hotkey.

Lists in Java Attempt #2

Let's try to make a copy of the code below using IntelliJ.

- The code below also won't compile.
- This time IntelliJ complains that "List is abstract, cannot be instantiated".
- The problem is that we need to pick a specific type of List to create.
 - (More in a moment!)

```
L = []  
L.append("a")  
L.append("b")  
L.append("c")  
print(L)
```

```
['a', 'b', 'c']
```

```
import java.util.List;  
  
void main() {  
    List<String> L = new List();  
}
```

Lists in Java Attempt #3

This code finally compiles.

- Note: This code is very old school Java (circa version 4.0 from 2004), and we'll update it to be more modern a bit later.
- Now that we have a list, we can start adding things to it.

```
L = []  
L.append("a")  
L.append("b")  
L.append("c")  
print(L)
```

```
['a', 'b', 'c']
```

```
import java.util.ArrayList;  
import java.util.List;  
  
void main() {  
    List L = new ArrayList();  
}
```

We'll talk about this distinction shortly.

List Example in Java

We've written the equivalent Java program!

- Note: This code is very old school Java (circa version 4.0 from 2004), and we'll update it to be more modern a bit later.

Now let's reflect on that distinction between List and ArrayList.

```
L = []  
L.append("a")  
L.append("b")  
L.append("c")  
print(L)
```

```
['a', 'b', 'c']
```

```
import java.util.ArrayList;  
import java.util.List;  
  
void main() {  
    List L = new ArrayList();  
    L.add("a");  
    L.add("b");  
    L.add("c");  
    System.out.println(L);  
}
```

Abstract Data Types vs. Concrete Implementations

Lecture 2, CS61B, Spring 2026

Classes in Java

- Defining and Instantiating Classes
- Class Terminology
- Static vs. Instance Members
- `public`
- `public static void main(String[] args)`

Lists and Arrays

- Lists in Java 4.0
- **Abstract Data Types vs. Concrete Implementations**

List Example in Java

Let's reflect on that distinction between List and ArrayList.

```
L = []  
L.append("a")  
L.append("b")  
L.append("c")  
print(L)
```

```
['a', 'b', 'c']
```

```
import java.util.ArrayList;  
import java.util.List;  
  
void main() {  
    List L = new ArrayList();  
    L.add("a");  
    L.add("b");  
    L.add("c");  
    System.out.println(L);  
}
```

Alternate Types of List

Java has other types of Lists. Let's take a peek:

- <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/List.html>
- Most common type of **List** is **ArrayList**, but **LinkedList** is also used sometimes.

```
L = []  
L.append("a")  
L.append("b")  
L.append("c")  
print(L)
```

```
['a', 'b', 'c']
```

```
import java.util.ArrayList;  
import java.util.List;  
  
void main() {  
    List L = new ArrayList();  
    ...  
}
```

Alternate Types of List

Java has other types of Lists. Let's take a peek:

- <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/List.html>
- Most common type of **List** is **ArrayList**, but **LinkedList** is also used sometimes.

```
L = []  
L.append("a")  
L.append("b")  
L.append("c")  
print(L)
```

```
['a', 'b', 'c']
```

```
import java.util.ArrayList;  
import java.util.List;  
  
void main() {  
    List L = new LinkedList();  
    ...  
}
```


List

In Python, there is no distinction between the abstract idea of a list and an actual list.

- A list is a list is a list. Programmer has no choice of what type of list.

In Java, there are many types of lists.

- Programmer has to decide which one.

```
L = []  
L.append("a")  
L.append("b")  
L.append("c")  
print(L)
```

```
['a', 'b', 'c']
```

```
import java.util.ArrayList;  
import java.util.List;  
  
void main() {  
    List L = new LinkedList();  
    ...  
}
```

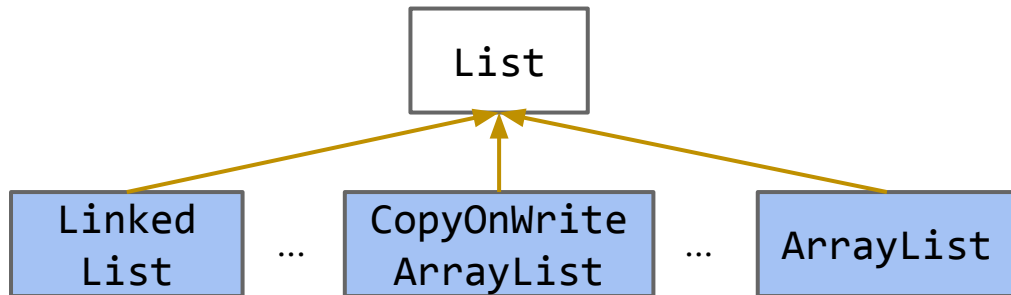
Abstract Data Types vs. Concrete Implementations

Another term used for List in Java is “Abstract Data Type”.

- Any list is guaranteed to have at least the operations in <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/List.html>.

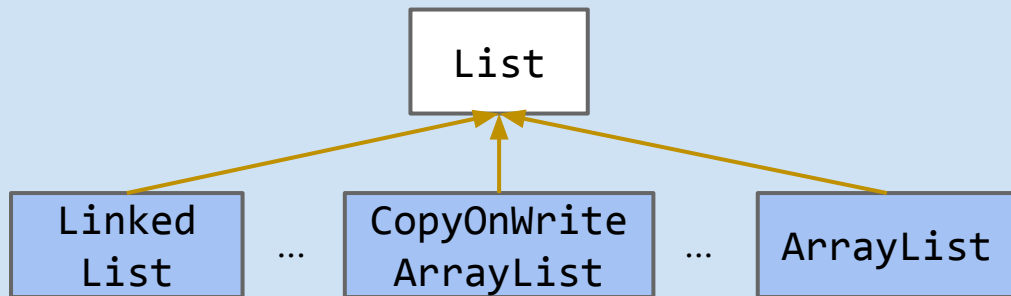
Each implementation, e.g. LinkedList, is known as a “Concrete Implementation”.

- Code for different types of list may be radically different.
- All concrete implementations have at least the operations guaranteed by every List (at the link above).



Abstract Data Types

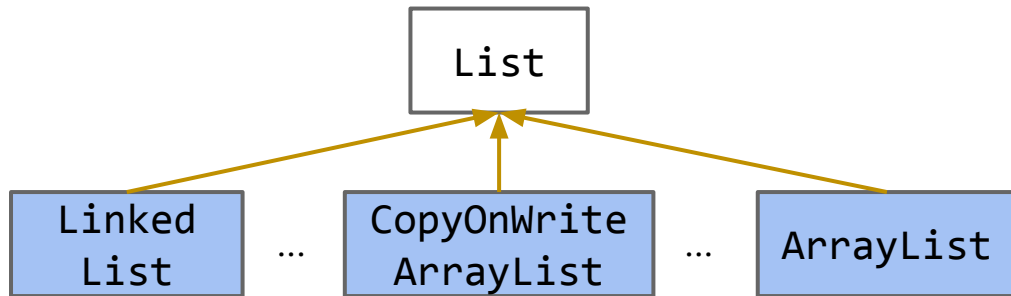
Why bother having multiple implementations? What do you think?



Why bother having multiple concrete implementations of an abstract data type?

- May have better performance in certain cases. Example:
 - LinkedLists are very fast at removing the front item. ArrayLists are very slow to remove the front item.
- May have additional operations. Example:
 - The Stack implementation of List has a “push” and “pop” operation.

We'll come to explore this concept in much more detail next week.



For Next Time

Today, we've only covered how list usage looked back in the very distant Java 4.0 days.

- In lecture 3, we'll start by seeing how lists are used today.

```
L = []  
L.append("a")  
L.append("b")  
L.append("c")  
print(L)
```

```
import java.util.ArrayList;  
import java.util.List;  
  
void main() {  
    List L = new ArrayList();  
    L.add("a");  
    L.add("b");  
    L.add("c");  
    System.out.println(L);  
}
```

Todo

That's all for today.

- Make sure to finish HW1 and HW2!